

SKILL WEEK – 11

TASK1: To connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
shreyaj@LAPTOP-FT6VVM4D: ~$ mkdir -p ~/Skill11
shreyaj@LAPTOP-FT6VVM4D: ~/Skill11$ cd ~/Skill11
shreyaj@LAPTOP-FT6VVM4D: ~/Skill11$ nano calculator.h
shreyaj@LAPTOP-FT6VVM4D: ~/Skill11$ nano calculator.c
shreyaj@LAPTOP-FT6VVM4D: ~/Skill11$ nano main.c
shreyaj@LAPTOP-FT6VVM4D: ~/Skill11$ nano main.c
shreyaj@LAPTOP-FT6VVM4D: ~/Skill11$ gcc main.c calculator.c -o program
shreyaj@LAPTOP-FT6VVM4D: ~/Skill11$ ./program
Addition = 15
Subtraction = 5
shreyaj@LAPTOP-FT6VVM4D: ~/Skill11$ |
```

```
GNU nano 8.7.1
#ifdef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);

#endif
```

```
GNU nano 8.7.1
#include "calculator.h"

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}
```

```
shreyaj@LAPTOP-FT6VVM4D: × + v
GNU nano 8.7.1 main.c
#include <stdio.h>
#include "calculator.h"

int main() {
    int a = 10;
    int b = 5;

    printf("Addition = %d\n", add(a, b));
    printf("Subtraction = %d\n", subtract(a, b));

    return 0;
}
```

TASK2: To detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

```
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ nano error_handling.c
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ gcc error_handling.c -o error_handling
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ ./error_handling
Enter two integers: 10 2
Result = 5
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ ./error_handling
Enter two integers: 10 0
ERROR: Division by zero is not allowed.
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ ./error_handling
Enter two integers: abc xyz
ERROR: Invalid input. Please enter integers.
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ ./error_handling > output.log 2>&1
cat output.log
```

```
shreyaj@LAPTOP-FT6VVM4D: × + v
GNU nano 8.7.1 error_handling.c
#include <stdio.h>

int main() {
    int a, b;

    printf("Enter two integers: ");
    if (scanf("%d %d", &a, &b) != 2) {
        printf("ERROR: Invalid input. Please enter integers.\n");
        return 1;
    }

    if (b == 0) {
        printf("ERROR: Division by zero is not allowed.\n");
        return 1;
    }

    printf("Result = %d\n", a / b);
    return 0;
}
```

TASK3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
shreyaj@LAPTOP-FT6VVM4D: ~$ nano leak.c
shreyaj@LAPTOP-FT6VVM4D: ~$ gcc -g leak.c -o leak
valgrind --leak-check=full ./leak
==1005== Memcheck, a memory error detector
==1005== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1005== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1005== Command: ./leak
==1005==
Value: 42
==1005==
==1005== HEAP SUMMARY:
==1005==    in use at exit: 40 bytes in 1 blocks
==1005==    total heap usage: 2 allocs, 1 frees, 1,064 bytes allocated
==1005==
==1005== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==1005==    at 0x4850858: malloc (vg_replace_malloc.c:447)
==1005==    by 0x400117E: main (leak.c:5)
==1005==
==1005== LEAK SUMMARY:
==1005==    definitely lost: 40 bytes in 1 blocks
==1005==    indirectly lost: 0 bytes in 0 blocks
==1005==    possibly lost: 0 bytes in 0 blocks
==1005==    still reachable: 0 bytes in 0 blocks
==1005==    suppressed: 0 bytes in 0 blocks
==1005==
==1005== For lists of detected and suppressed errors, rerun with: -s
==1005== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
shreyaj@LAPTOP-FT6VVM4D: ~$ nano leak.c
shreyaj@LAPTOP-FT6VVM4D: ~$ gcc -g leak.c -o leak
valgrind --leak-check=full ./leak
==1041== Memcheck, a memory error detector
==1041== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1041== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1041== Command: ./leak
==1041==
Value: 42
==1041==
==1041== HEAP SUMMARY:
==1041==    in use at exit: 0 bytes in 0 blocks
==1041==    total heap usage: 2 allocs, 2 frees, 1,064 bytes allocated
==1041==
==1041== All heap blocks were freed -- no leaks are possible
==1041==
==1041== For lists of detected and suppressed errors, rerun with: -s
==1041== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

```
GNU nano 8.7.1 leak.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(sizeof(int) * 10);
    *ptr = 42;

    printf("Value: %d\n", *ptr);

    free(ptr); // Clean up dynamically allocated memory
    return 0;
}
```

TASK4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

```
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ nano test_script.sh
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ ^C
shreyaj@LAPTOP-FT6VVM4D:~/Skill11$ chmod +x test_script.sh
./test_script.sh
Test Report
=====
Starting automated tests for error_handling...
PASS: Valid Input (10/2)
PASS: Division by Zero
PASS: Invalid Syntax
=====
Testing complete.
```

```
shreyaj@LAPTOP-FT6VVM4D: × + v
GNU nano 8.7.1 test_script.sh
#!/bin/bash

echo "Test Report" > test_report.txt
echo "===== " >> test_report.txt
echo "Starting automated tests for error_handling..." >> test_report.txt

test_case() {
    local input_data="$1"
    local expected_output="$2"
    local test_name="$3"

    actual_output=$(echo "$input_data" | ./error_handling)

    if [ "$actual_output" = "$expected_output" ]; then
        echo "PASS: $test_name" >> test_report.txt
    else
        echo "FAIL: $test_name" >> test_report.txt
        echo " Expected: $expected_output" >> test_report.txt
        echo " Actual: $actual_output" >> test_report.txt
    fi
}

test_case "10 2" "Enter two integers: Result = 5" "Valid Input (10/2)"
test_case "10 0" "Enter two integers: ERROR: Division by zero is not allowed." "Division by Zero"
test_case "abc xyz" "Enter two integers: ERROR: Invalid input. Please enter integers." "Invalid Syntax"

echo "===== " >> test_report.txt
echo "Testing complete." >> test_report.txt

cat test_report.txt
```